TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced - Important Coding Problems

PROBLEM, ALGORITHM & EXPLANATION

SOLUTION (JAVA)

Q1. IMPLEMENT STACK USING ARRAY

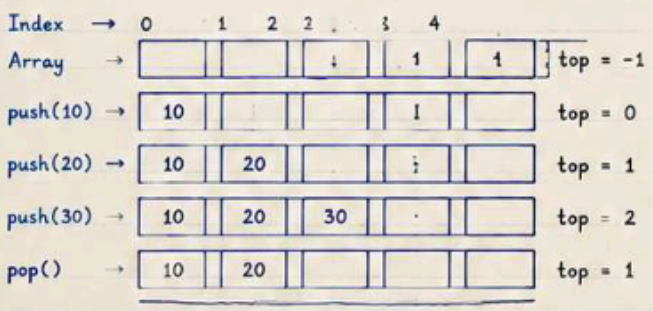
Problem Statement :

Design a stack that supports push, pop, top and isEmpty operations using array.

Algorithm :

1. Create an array of size N.
2. Initialize top = -1.
3. Push(x) :
   - → If top == N-1, Stack is full.
   - → Else, top = top + 1 and arr[top] = x.
4. Pop() :
   - → If top == -1, Stack is empty.
   - → Else, return arr[top] and top = top - 1.
5. Top() :
   - → If top == -1, return -1.
   - → Else, return arr[top].
6. isEmpty() :
   - → Return true if top == -1 else false.

Dry Run (Push 10, 20, 30 ; Pop)



Complexity :

- Push : O(1)
- Pop : O(1)
- Top : O(1)
- Space : O(N)

Key Point :

Stack follows LIFO (Last In First Out) order.

```java
class ArrayStack {
    private int[] arr;
    private int top;
    private int capacity;

    public ArrayStack(int n) {
        capacity = n;
        arr = new int[capacity];
        top = -1;
    }

    public void push(int x) {
        if (top == capacity - 1) {
            System.out.println("Stack Underflow");
            return;
        }
        top++;
        arr[top] = x;
    }

    public int pop() {
        if (top == -1) {
            System.out.println("Stack Underflow");
            return -1;
        }
        int x = arr[top];
        top--;
        return x;
    }

    public int top() {
        if (top == -1) return -1;
        return arr[top];
    }

    public boolean isEmpty() {
        return top == -1;
    }
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced - Important Coding Problems

PROBLEM, ALGORITHM & EXPLANATION

Q2. IMPLEMENT QUEUE USING ARRAY

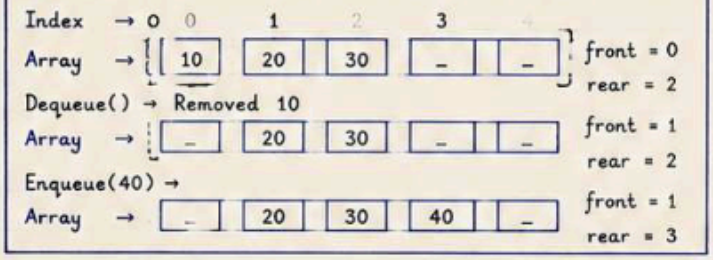
Problem Statement :

Design a queue that supports enqueue, dequeue, front, rear and isEmpty operations using array.

Algorithm :

1. Create an array of size N.
2. Initialize front = rear = -1.
3. Enqueue(x) :
   - → If rear == N-1, Queue is full.
   - → Else
     - If front == -1, set front = 0.
     - Set rear = rear + 1 and arr[rear] = x.
4. Dequeue() :
   - → If front == -1 or front > rear, Queue is empty.
   - → Else
     - x = arr[front];
     - If front == rear, set front = rear = -1.
     - Else front = front + 1.
     - Return x.
5. Front() :
   - → If queue empty, return -1.
   - → Else return arr[front].
6. Rear() :
   - → If queue empty, return -1.
   - → Else return arr[rear].
7. isEmpty() :
   - → Return true if front == -1 else false.
8. isFull() :
   - → Return true if rear == N-1 else false.

Dry Run (Enqueue 10, 20, 30 ; Dequeue ; Enqueue 40)



SOLUTION (JAVA)

```java
class ArrayQueue {
    private int[] arr;
    private int front, rear, size;
    private int capacity;

    public ArrayQueue(int n) {
        capacity = n;
        arr = new int[capacity];
        front = rear = -1;
    }

    // Enqueue
    public void enqueue(int x) {
        if (rear == capacity -1) {
            System.out.println("Queue Overflow");
            return;
        }
        if (front == -1) front = 0;
        rear++;
        arr[rear] = x;
    }

    // Dequeue
    public int dequeue() {
        if (front == -1 || front > rear) {
            System.out.println("Queue Underflow");
            return -1;
        }
        int x = arr[front];
        if (front == rear) {
            front = rear = -1;
        } else {
            front++;
        }
        return x;
    }
    // Front
    public int front() {
        if (front == -1 || front > rear) return -1;
        return arr[front];
    }
    // Rear
    public int rear() {
        if (front == -1 || front > rear) return -1;
        return arr[rear];
    }
    // isEmpty
    public boolean isEmpty() {
        return front == -1;
    }
    // isFull
    public boolean isFull() {
        return rear == capacity - 1;
    }
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

Q3. IMPLEMENT STACK USING LINKED LIST

Problem Statement :

Design a stack that supports push, pop, top and isEmpty operations using linked list.

Algorithm (Short) :

1. Use a singly linked list.
2. New node is always added at head.
3. Push → Insert new node at head.
4. Pop → Delete node from head.
5. Top → Return data of head.
6. isEmpty → Check if head is null.

```java
class LinkedListStack {
  class Node {
      int data;
      Node next;
      Node(int d){ data = d; next = null; }
  }
  private Node head;

  // Push
  public void push(int x){
     Node newNode = new Node(x);
     newNode.next = head;
     head = newNode;
  }
  // Pop
  public int pop(){
     if(head == null) throw new RuntimeException("Stack Underflow");
     int x = head.data;
     head = head.next;
     return x;
  }
  // Top
  public int top(){
     if(head == null) throw new RuntimeException("Stack Underflow");
     return head.data;
  }
  // isEmpty
  public boolean isEmpty(){
     return head == null;
  }
}
```

Q4. IMPLEMENT QUEUE USING LINKED LIST

Problem Statement :

Design a queue that supports enqueue, dequeue, front, rear and isEmpty operations using linked list.

Algorithm (Short) :

1. Use a singly linked list with two pointers front and rear.
2. Enqueue → Insert new node at rear.
3. Dequeue → Delete node from front.
4. Front → Return data at front.
5. Rear → Return data at rear.
6. isEmpty → Check if front is null.

```java
class LinkedListQueue {
  class Node {
      int data;
      Node next;
      Node(int d){ data = d; next = null; }
  }
  private Node front, rear;
  // Enqueue
  public void enqueue(int x){
    Node newNode = new Node(x);
    if(rear == null){
       front = rear = newNode;
       return;
    }
    rear.next = newNode;
    rear = newNode;
  }
  // Dequeue
  public int dequeue(){
    if(front == null) throw new
    RuntimeException("Queue Underflow");
    int x = front.data;
    front = front.next;
    if(front == null) rear = null;
    return x;
  }
  // Front
  public int front(){
    if(front == null) throw new
    RuntimeException("Queue Underflow");
    return front.data;
  }
  // Rear
  public int rear(){
    if(rear == null) throw new
    RuntimeException("Queue Underflow");
    return rear.data;
  }
  // isEmpty
  public boolean isEmpty(){
    return front == null;
  }
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced – Important Coding Problems

PROBLEM, ALGORITHM & EXPLANATION	SOLUTION (JAVA)

Q6. REVERSE A STACK
Problem Statement :
Reverse a given stack using recursion.
Algorithm (Short) :
1. If stack is empty, return.
2. Pop the top element and store it.
3. Recursively reverse the remaining stack.
4. Insert the stored element at the bottom of the stack.
5. Repeat until stack is reversed.
	`// Function to reverse stack`
	`public void reverse(Stack<Integer> st){`
	`  if(st.isEmpty()) return;`
	`  int top = st.pop();`
	`  reverse(st);`
	`  insertAtBottom(st, top);`
	`}`
	`// Insert element at bottom`
	`private void insertAtBottom(Stack<Integer> st, int x){`
	`  if(st.isEmpty()){`
	`  st.push(x);`
	`  return;`
	`  }`
	`  int top = st.pop();`
	`  insertAtBottom(st, x);`
	`  st.push(top);`
	`}`

Q7. REVERSE A QUEUE
Problem Statement :
Reverse a given queue.
Algorithm (Short) :
1. If queue is empty, return.
2. Dequeue the front element and store it.
3. Recursively reverse the remaining queue.
4. Enqueue the stored element at the rear.
5. Repeat until queue is reversed.
	`// Function to reverse queue`
	`public void reverse(Queue<Integer> q){`
	`  if(q.isEmpty()) return;`
	`  int front = q.poll();`
	`  reverse(q);`
	`  q.offer(front);`
	`}`

Q8. CHECK BALANCED PARENTHESES
Problem Statement :
Given an expression containing brackets '()', '{}', '[]', check if it is balanced.
Algorithm (Short) :
1. Traverse the expression.
2. Push opening brackets onto stack.
3. For closing bracket, check if stack is empty or the top doesn't match -> not balanced.
4. If stack is empty at end -> balanced, else not balanced.
	`// Function to check balanced parentheses`
	`public boolean isBalanced(String s){`
	`  Stack<Character> st = new Stack<>();`
	`  for(char ch : s.toCharArray()){`
	`  if(ch == '(' || ch == '{' || ch == '[')`
	`  st.push(ch);`
	`  else{`
	`  if(st.isEmpty()) return false;`
	`  char top = st.pop();`
	`  if((ch == ')' && top != '(') ||`
	`  (ch == '}' && top != '{') ||`
	`  (ch == ']' && top != '['))`
	`  return false;`
	`  }`
	`  }`
	`  return st.isEmpty();`
	`}`

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced – Important Coding Problems

PROBLEM, ALGORITHM & EXPLANATION	SOLUTION (JAVA)

Q9. VALID PARENTHESES

Problem Statement :

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

Algorithm (Short) :

1. Traverse the string.
2. Push opening brackets onto stack.
3. For closing bracket, check if stack is empty or top doesn't match.
4. If all brackets are matched and stack is empty at end ⇒ valid.

```java
// Function to check valid parentheses
public boolean isValid(String s){
    Stack<Character> st = new Stack<>();
    for(char ch : s.toCharArray()){
        if(ch == '(' || ch == '{' || ch == '[')
            st.push(ch);
        else{
            if(st.isEmpty()) return false;
            char top = st.pop();
            if((ch == ')' && top != '(') ||
               (ch == '}' && top != '{') ||
               (ch == ']' && top != '[')|
               return false;
        }
    }
    return st.isEmpty();
}
```

Q10. NEXT GREATER ELEMENT

Problem Statement :

Given an array arr, for each element find the next greater element on its right side.
If no such element exists, return -1 for that index.

Algorithm (Short) :

1. Traverse array from right to left.
2. Maintain a stack to store greater elements.
3. Pop elements smaller or equal to current.
4. Top of stack is next greater element.
5. Push current element onto stack.

```java
// Function to find next greater element
public int[] nextGreater(int[] arr){
    int n = arr.length;
    int[] res = new int[n];
    Stack<Integer> st = new Stack<>();
    for(int i = n-1; i >= 0; i--){
        while(!st.isEmpty() && st.peek() <= arr[i])
            st.pop();
        res[i] = st.isEmpty() ? -1 : st.peek();
        st.push(arr[i]);
    }
    return res;
}
```

Q11. NEXT SMALLER ELEMENT

Problem Statement :

Given an array arr, for each element find the next smaller element on its right side.
If no such element exists, return -1 for that index.

Algorithm (Short) :

1. Traverse array from right to left.
2. Maintain a stack to store smaller elements.
3. Pop elements greater or equal to current.
4. Top of stack is next smaller element.
5. Push current element onto stack.

```java
// Function to find next smaller element
public int[] nextSmaller(int[] arr){
    int n = arr.length;
    int[] res = new int[n];
    Stack<Integer> st = new Stack<>();
    for(int i = n-1; i >= 0; i--){
        while(!st.isEmpty() && st.peek() >= arr[i])
            st.pop();
        res[i] = st.isEmpty() ? -1 : st.peek();
        st.push(arr[i]);
    }
    return res;
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced – Important Coding Problems

PROBLEM, ALGORITHM & EXPLANATION	SOLUTION (JAVA)

Q12. PREVIOUS GREATER ELEMENT

Problem Statement :

Given an array arr, for each element find the previous greater element on its left side.
If no such element exists, return -1 for that index.

Algorithm (Short) :

1. Traverse array from left to right.
2. Maintain a stack to store greater elements.
3. Pop elements smaller or equal to current.
4. Top of stack is previous greater element.
5. Push current element onto stack.

```java
// Function to find previous greater element
public int[] previousGreater(int[] arr){
    int n = arr.length;
    int[] res = new int[n];
    Stack<Integer> st = new Stack<>();
    for(int i = 0; i < n; i++){
        while(!st.isEmpty() && st.peek() <= arr[i])
            st.pop();
        res[i] = st.isEmpty() ? -1 : st.peek();
        st.push(arr[i]);
    }
    return res;
}
```

Q13. PREVIOUS SMALLER ELEMENT

Problem Statement :

Given an array arr, for each element find the previous smaller element on its left side.
If no such element exists, return -1 for that index.

Algorithm (Short) :

1. Traverse array from left to right.
2. Maintain a stack to store smaller elements.
3. Pop elements greater or equal to current.
4. Top of stack is previous smaller element.
5. Push current element onto stack.

```java
// Function to find previous smaller element
public int[] previousSmaller(int[] arr){
    int n = arr.length;
    int[] res = new int[n];
    Stack<Integer> st = new Stack<>();
    for(int i = 0; i < n; i++){
        while(!st.isEmpty() && st.peek() >= arr[i])
            st.pop();
        res[i] = st.isEmpty() ? -1 : st.peek();
        st.push(arr[i]);
    }
    return res;
}
```

Q14. REMOVE ADJACENT DUPLICATES FROM STRING

Problem Statement :

Given a string s, remove adjacent duplicate characters repeatedly until no such pair exists.
Return the final string.

Algorithm (Short) :

1. Traverse each character in string.
2. Use a stack to store characters.
3. If top of stack is same as current char, pop it (remove pair).
4. Else, push current char onto stack.
5. Build string from stack.

```java
// Function to remove adjacent duplicates
public String removeDuplicates(String s){
    Stack<Character> st = new Stack<>();
    for(char ch : s.toCharArray()){
        if(!st.isEmpty() && st.peek() == ch)
            st.pop();
        else{
            st.push(ch);
        }
    }
    StringBuilder sb = new StringBuilder();
    while(!st.isEmpty())
        sb.append(st.pop());
    return sb.reverse().toString();
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced – Important Coding Problems

PROBLEM, ALGORITHM & EXPLANATION	SOLUTION (JAVA)

Q15. INFIX TO POSTFIX CONVERSION

Problem Statement :

Convert a given infix expression to its postfix expression.

Algorithm (Short) :

1. Scan the expression from left to right.
2. If operand, add to result.
3. If '(', push to stack.
4. If operator, pop from stack and add to result while top has higher or equal precedence.
5. If ')', pop and add to result until '(' is found.
6. After scanning, pop remaining operators.

```
// Function to convert infix to postfix
public String infixToPostfix(String exp) {
    StringBuilder res = new StringBuilder();
    Stack<Character> st = new Stack<>();
    for(char ch : exp.toCharArray()){
        if(Character.isLetterOrDigit(ch){
            res.append(ch);
        }else if(ch == '('){
            st.push(ch);
        }else if(ch == ')'){
            while(!st.isEmpty() && st.peek() != '('
            {  res.append(st.pop());
            if(!st.isEmpty()) st.pop();     // pop '('
        }else{  // operator
            while(!st.isEmpty() && precedence(st.peek()) >= preccdence(ch))
              }  res.append(st.pop());
              st.push(ch);
            }
    }
    while(!st.isEmpty()) res.append(st.pop());
    return res.toString();
}
private int precedence(char ch){
    if(ch=='^') return 3;
    if(ch=='*' || ch=='/') return 2;
    if(ch=='+' || ch=='-') return 1;
    return -1;
}
```

Q16. INFIX TO PREFIX CONVERSION

Problem Statement :

Convert a given infix expression to its prefix expression.

Algorithm (Short) :

1. Reverse the infix expression.
2. Replace '(' with ')' and vice versa.
3. Convert the modified expression to postfix.
4. Reverse the result to get prefix.

```
// Function to convert infix to prefix
public String infixToPreffix(String exp) {
    // Step 1: Reverse expression
    StringBuilder sb = new StringBuilder(exp);
    sb.reverse();
    String revExp = sb.toString();

    // Step 2: Replace parentheses
    revExp = revExp.replace('(', '#').replace(')', '(').replace('#', ')');

    // Step 3: Convert reversed expression to postfix
    String postfix = infixToPostfix(revExp);    // use Q15 function

    // Step 4: Reverse postfix to get prefix
    return new StringBuilder(postfix).reverse().toString();
}
```

Q17. POSTFIX EXPRESSION EVALUATION

Problem Statement :

Evaluate a given postfix expression and return the result.

Algorithm (Short) :

1. Scan the expression from left to right.
2. If operand, push it to stack.
3. If operator, pop two operands, apply operator (second popped is left operand).
4. Push the result back to stack.
5. Final element in stack is the answer.

```
// Function to evaluate postfix expression
public int evaluatePostfix(String exp) {
    Stack<Integer> st = new Stack<>();
    for(char ch : exp.toCharArray()){
        if(Character.isDigit(ch)) {
            st.push(ch - '0');
        }else{
            int b = st.pop();     // right operand
            int a = st.pop();     // left operand
            int res = 0;
            switch(ch){
                case '+': res = a + b; break;
                case '-': res = a - b; break;
                case '*': res = a * b; break;
                case '/': res = a / b; break;
            }
            st.push(res);
    }
    return st.pop();
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced - Important Coding Problems

PROBLEM, ALGORITHM & EXPLANATION	SOLUTION (JAVA)

Q18. POSTFIX EXPRESSION EVALUATION

Problem Statement :

Evaluate a given postfix expression and return the result.

Algorithm (Short) :

1. Scan the expression from left to right.
2. If operand, push it onto stack.
3. If operator, pop two operands (b, a). Apply operator a (op) b and push result.
4. After scanning complete expression, top of stack is the answer.

```java
// Function to evaluate postfix expression
public int evaluatePostfix(String exp) {
    Stack<Integer> st = new Stack<>();
    for(char ch : exp.toCharArray()){
        if(Character.isDigit(ch)){
            st.push(ch - '0');
        } else {
            int b = st.pop();   // right operand
            int a = st.pop();   // left operand
            int res = 0;
            switch(ch){
                case '+': res = a + b; break;
                case '-': res = a - b; break;
                case '*': res = a * b; break;
                case '/': res = a / b; break;
            }
            st.push(res);
        }
    }
    return st.pop();    // final result
}
```

Q19. MIN STACK

Problem Statement :

Design a stack that supports push, pop, top and retrieving the minimum element in O(1) time.

Algorithm (Short) :

1. Use two stacks: main stack and min stack.
2. Push element to main stack.
3. If min stack is empty or current <= min top, push to min stack.
4. On pop, if popped element is equal to min stack top, pop from min stack also.
5. GetMin() is top of min stack.

```java
// Min Stack using two stacks
class MinStack {
    Stack<Integer> st = new Stack<>();
    Stack<Integer> minSt = new Stack<>();

    public void push(int x){
        st.push(x);
        if(minSt.isEmpty() || x <= minSt.peek())
            minSt.push(x);
    }
    public void pop(){
        int val = st.pop();
        if(val == minSt.peek()) minSt.pop();
    }
    public int top(){
        return st.peek();
    }
    public int getMin(){
        return minSt.peek();
    }
}
```

Q20. IMPLEMENT QUEUE USING TWO STACKS

Problem Statement :

Implement a queue using two stacks to support enqueue, dequeue, front and isEmpty.

Algorithm (Short) :

1. Use two stacks: s1 (for enqueue), s2 (for dequeue).
2. Enqueue: Push element to s1.
3. Dequeue/Front: If s2 is empty, pop all elements from s1 and push to s2. Then pop/peek from s2.
4. isEmpty: Both stacks are empty.

```java
// Queue using two stacks
class MyQueue {
    Stack<Integer> s1 = new Stack<>();
    Stack<Integer> s2 = new Stack<>();
  // Enqueue
  public void push(int x){
      s1.push(x);
  }
  // Dequeue
  public int pop(){
      if(s2.isEmpty()) transfer();
      return s2.pop();
  }
  // Front
  public int peek(){
      if(s2.isEmpty()) transfer();
      return s2.peek();
  }
  // Helper
  private void transfer(){
      while(!s1.isEmpty()) s2.push(s1.pop());
  }
  public boolean isEmpty(){
      return s1.isEmpty() && s2.isEmpty();
  }
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced - Important Coding Problems

Q21. IMPLEMENT STACK USING TWO QUEUES

Problem Statement :

Implement a stack using two queues.

Algorithm (Short) :

1. Use two queues q1 and q2.
2. For push(x):
   - → Add x to q2.
   - → Move all elements from q1 to q2.
   - → Swap q1 and q2.
3. For pop(): Remove element from q1.
4. For top(): Return front of q1.
5. isEmpty(): Check if q1 is empty.

```
class MyStack {
   Queue<Integer> q1 = new LinkedList<>();
   Queue<Integer> q2 = new LinkedList<>();

   // Push element
   public void push(int x) {
      q2.offer(x);
      while(!q1.isEmpty()) q2.offer(q1.poll());
      Queue<Integer> temp = q1;
      q1 = q2;
      q2 = temp;
   }

   public int pop() { return q1.poll(); }

   public int top() { return q1.peek(); }

   public boolean isEmpty() { return q1.isEmpty(); }
}
```

Q22. SORT A STACK USING RECURSION

Problem Statement :

Sort the elements of a stack using recursion (largest at top).

Algorithm (Short) :

1. If stack is empty or has one element, return.
2. Pop the top element and sort remaining stack.
3. Insert the popped element in sorted order using recursive helper.
4. Use helper function to insert element at correct position.

```
// Sort stack
public void sort(Stack<Integer> st) {
   if(st.isEmpty() || st.size() == 1) return;
   int top = st.pop();
   sort(st);
   insertSorted(st, top);
}
// Insert element in sorted stack
private void insertSorted(Stack<Integer> st, int x) {
   if(st.isEmpty() || x > st.peek()) {
      st.push(x);
      return;
   }
   int temp = st.pop();
   insertSorted(st, x);
   st.push(temp);
}
```

Q23. STOCK SPAN PROBLEM

Problem Statement :

Given daily stock prices, for each day find span = number of consecutive days (including today) with price less than or equal to today's price.

Algorithm (Short) :

1. Use a stack to store indices.
2. For each day, pop while price[stack.top] <= current price.
3. Span = i+1 if stack is empty else i - stack.top.
4. Push current index to stack.

```
// Function to find stock span
public int[] stockSpan(int[] price) {
   int n = price.length;
   int[] span = new int[n];
   Stack<Integer> st = new Stack<>();
   span[0] = 1;
   st.push(0);

   for(int i = 1; i < n; i++) {
      while(!st.isEmpty() && price[st.peek()] <= price[i])
            st.pop();
      span[i] = st.isEmpty() ? (i + 1) : (i - st.peek());
      st.push(i);
   }
   return span;
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced – Important Coding Problems

Q24. FIRST NON-REPEATING CHARACTER IN STREAM

Problem Statement :

Given a stream of characters, print the first non-repeating character at every step.

Algorithm (Short) :

1. Maintain a queue for characters.
2. Maintain a frequency array (size 256).
3. For each character:
   - → Enqueue the character.
   - → Increment its frequency.
   - → Remove characters from front of queue while their frequency > 1.
   - → Front of queue is first non-repeating character, else NONE.

```
// Function to process stream
public void firstNonRepeating(String stream){
    Queue<Character> q = new LinkedList<>();
    int[] freq = new int[256];
    for(char ch : stream.toCharArray()){
        q.offer(ch);
        freq[ch]++;
        while(!q.isEmpty() && freq[q.peek()] > 1)
            q.poll();
        if(q.isEmpty())
            System.out.print(-1 + " ");
        else
            System.out.print(q.peek() + " ");
    }
}
```

Q25. GENERATE BINARY NUMBERS 1 TO N

Problem Statement :

Generate binary numbers from 1 to N.

Algorithm (Short) :

1. Use a queue to store binary strings.
2. Start by pushing "1".
3. For each number:
   - → Dequeue front.
   - → Print it.
   - → Enqueue front + "0" and front + "1".

```
// Function to generate binary numbers
public void generateBinary(int n) {
    Queue<String> q = new LinkedList<>();
    q.offer("1");
    for(int i = 1; i <= n; i++){
        String front = q.poll();
        System.out.print(front + " ");
        q.offer(front + "0");
        q.offer(front + "1");
    }
}
```

Q26. SLIDING WINDOW MAXIMUM

Problem Statement :

Given an array and window size k, find maximum in each sliding window.

Algorithm (Short) :

1. Use Deque (stores indices) to store useful elements.
2. Remove indices out of current window.
3. Remove smaller elements from back.
4. Front of deque is max of current window.
5. Slide window and repeat.

```
// Function to find maximum in sliding windows
public int[] maxSlidingWindow(int[] arr, int k) {
    int n = arr.length;
    int[] res = new int[n - k + 1];
    Deque<Integer> dq = new LinkedList<>();
    for(int i = 0; i < n; i++) {
        // Remove out of window
        if(!dq.isEmpty() && dq.peekFirst() <= i - k)
            dq.pollFirst();
        // Remove smaller elements
        while(!dq.isEmpty() && arr[dq.peekLast()] <= arr[i])
            dq.pollLast();
        dq.offerLast(i);
        // Store result
        if(i >= k - 1)
            res[i - k + 1] = arr[dq.peekFirst()];
    }
    return res;
}
```

TOP 30 STACK AND QUEUE PROBLEMS

MOST ASKED INTERVIEW PROBLEMS

From Basics to Advanced - Important Coding Problems

Q27. EVALUATE PREFIX EXPRESSION

Problem Statement :

Evaluate a given prefix expression and return the result.

Algorithm (Short) :

1. Scan from right to left.
2. If operand, push it to stack.
3. If operator, pop two operands (first popped is left, second is right), apply operator and push result.
4. Final element in stack is answer.

```
// Function to evaluate prefix expression
public int evaluatePrefix(String exp) {
    Stack<Integer> st = new Stack<>();
    for(int i = exp.length() - 1; i >= 0; i--) {
        char ch = exp.charAt(i);
        if(Character.isDigit(ch)) {
            st.push(ch - '0');
        } else {
            int a = st.pop();   // left operand
            int b = st.pop();   // right operand
            int res = apply(ch, a, b);
            st.push(res);
        }
    }
    return st.pop();
}
private int apply(char op, int a, int b) {
    switch(op) {
        case '+': return a + b;   case '-': return a - b;
        case '*': return a * b;   case '/': return a / b;
    }
    return 0;
}
```

Q28. EVALUATE POSTFIX EXPRESSION

Problem Statement :

Evaluate a given postfix expression and return the result.

Algorithm (Short) :

1. Scan from left to right.
2. If operand, push it to stack.
3. If operator, pop two operands (first popped is right, second is left), apply operator and push result.
4. Final element in stack is answer.

```
// Function to evaluate postfix expression
public int evaluatePostfix(String exp) {
    Stack<Integer> st = new Stack<>();
    for(char ch : exp.toCharArray()) {
        if(Character.isDigit(ch)) {
            st.push(ch - '0');
        } else {
            int b = st.pop();   // right operand
            int a = st.pop();   // left operand
            int res = apply(ch, a, b);
            st.push(res);
        }
    }
    return st.pop();
}
private int apply(char op, int a, int b) {
    switch(op) {
        case '+': return a + b;   case '-': return a - b;
        case '*': return a * b;   case '/': return a / b;
    }
    return 0;
}
```

Q29. NEXT GREATER ELEMENT

Problem Statement :

For each element in the array, find the next greater element on its right side. If not exists, return -1 for that element.

Algorithm (Short) :

1. Traverse array from right to left.
2. Maintain a stack of elements.
3. Pop elements <= current.
4. Top of stack is next greater element.
5. Push current element onto stack.

```
// Function to find next greater element
public int[] nextGreater(int[] arr) {
    int n = arr.length;
    int[] res = new int[n];
    Stack<Integer> st = new Stack<>();
    for(int i = n - 1; i >= 0; i--) {
        while(!st.isEmpty() && st.peek() <= arr[i])
            st.pop();
        res[i] = st.isEmpty() ? -1 : st.peek();
        st.push(arr[i]);
    }
    return res;
}
```

Q30. IMPLEMENT DEQUE USING DLL

Problem Statement :

Implement Deque (double ended queue) using Doubly Linked List. Support insert/delete from both front and rear.

Algorithm (Short) :

1. Use doubly linked list with head and tail.
2. insertFront: add node at head.
3. insertRear: add node at tail.
4. deleteFront: remove node from head.
5. deleteRear: remove node from tail.
6. isEmpty: check if list is empty.

```
// Deque using Doubly Linked List
class Deque {
  class Node { int data; Node prev, next;
    Node(int d){ data = d; prev = next = null; } }
  Node head = null, tail = null;

  void insertFront(int x) {
    Node node = new Node(x);
    if(head == null) head = tail = node;
    else { node.next = head;  head.prev = node; head = node; }
  }
  void insertRear(int x) {
    Node node = new Node(x);
    if(tail == null) head = tail = node;
    else { tail.next = node;  node.prev = tail; tail = node; }
  }
  int deleteFront() {
    if(head == null) return -1;
    int val = head.data;
    if(head == tail) head = tail = null;
    else { head = head.next; head.prev = null; }
    return val;
  }
  int deleteRear() {
    if(tail == null) return -1;
    int val = tail.data;
    if(head == tail) head = tail = null;
    else { tail = tail.prev; tail.next = null; }
    return val;
  }
  boolean isEmpty() { return head == null; }
}
```